HotSOS

Mobile Safety Application

Capstone Project Documentation

Dhant Guenryan G. Rañada

Bachelor of Science in Information Technology

1. Project Overview

HotSOS is a real-time emergency response mobile application designed to improve the safety of adults and children. The application allows users to send one-tap SOS alerts, automatically share their precise location, and notify trusted responders through instant push notifications. The goal of HotSOS is to provide a fast, reliable emergency communication system that bridges the gap between victims and responders using modern cloud architecture and mobile hardware.

---

2. System Architecture & Technology Stack

The application utilizes a separated frontend and backend architecture to ensure scalability and real-time data synchronization.

2.1 Technologies Used

- **Frontend Framework:** React Native (Expo)
- **Programming Language:** JavaScript
- **Backend Platform:** Supabase (PostgreSQL, Authentication, Storage)
- **Real-time Engine:** Supabase WebSockets
- **Mapping Service:** Google Maps SDK
- **Package Manager:** npm

2.2 Core Dependencies

The project relies on several key libraries to interface with device hardware and backend services:

- **@supabase/supabase-js**: Handles database connections, authentication, and real-time listeners.

- **base64-arraybuffer**: A crucial utility that decodes image data to bypass Android's network restrictions when uploading profile avatars.
- **expo-image-picker**: Grants access to the device's camera and media gallery.
- **expo-location**: Captures high-accuracy GPS coordinates during an emergency.
- **expo-notifications**: Drives the push notification engine.
- **react-native-maps**: Integrates interactive map views for tracking.
- **react-native-safe-area-context**: Manages UI rendering around device notches and status bars.

---

3. Project Directory Structure

The codebase is modularized to separate UI components, application screens, and backend logic.

3.1 Frontend (safetyapp/)

- **/components (Reusable UI Elements)**
  - **SOSButton.js**: A specialized trigger component with built-in logic to prevent accidental activations (e.g., long-press requirements).
  - **ContactManager.js**: Handles the display, addition, and removal of users within the personal emergency network.
  - **EmergencyTracker.js**: The map-view engine that renders interactive markers for victims' real-time locations.
  - QuickAuth.js: Standardized input forms for login and registration.
- **/screens (Application Views)**
  - **HomeScreen.js**: The primary interface featuring the SOS Trigger. Handles hardware GPS coordinate capture and database logging.
  - **AlertsScreen.js**: A live feed utilizing Supabase Realtime to display incoming emergency alerts from the user's network instantly.
  - **LoginScreen.js** / RegisterScreen.js: User onboarding and secure authentication via Supabase Auth.

    - **ProfileScreen.js:** Manages user identity (Avatar, Nickname) and registers unique Expo Push Tokens for remote alerting. Uses Base64 encoding for Android upload compatibility.
    - **AddContactsScreen.js**: A privacy-focused search system allowing users to build a trusted network via nicknames.
- **/lib (Configurations)**
    - **supabase.js**: Centralized Supabase client configuration.
    - **notificationHelper.js**: Logic for managing device permissions and dispatching push alerts.

3.2 Backend (supabase/)

- **/migrations**: Version-controlled SQL scripts containing the database schema, relationships, and Row Level Security (RLS) policies.
- **config.toml**: CLI settings linking the local environment to the cloud project.

---

4. Database Schema (PostgreSQL)

The backend utilizes four primary tables. All relationships ensure that alerts and contacts are strictly associated with authenticated users via auth.users.

4.1 profiles

Stores additional user metadata.

- id (uuid, Primary Key, references auth.users)
- full_name (text)
- nickname (text)
- push_token (text)
- avatar_url (text)

4.2 alerts

Stores snapshot alert records created by users.

- id (uuid, Primary Key)
- user_id (uuid, references auth.users)
- contact_name (text)
- latitude (double precision)
- longitude (double precision)
- status (text, default 'active')

4.3 emergency_alerts

Stores continuous emergency records and live tracking details.

- id (uuid, Primary Key)
- user_id (uuid)
- victim_id (uuid, references auth.users)
- current_latitude / current_longitude (double precision)
- is_active (boolean)
- push_token (text)

4.4 emergency_contacts

Manages the trusted responder network.

- id (uuid, Primary Key)
- user_id (uuid, references profiles)
- contact_user_id (uuid, references profiles)

---

5. Technical Workflow: Notification Lifecycle

The application guarantees real-time delivery of emergency signals through the following pipeline:

1. **The Trigger (Client-Side):** The user presses the SOS button. expo-location retrieves precise device coordinates.
2. **Data Persistence:** The app performs an asynchronous INSERT into the Supabase alerts table.
3. **Foreground Synchronization:** Supabase Realtime detects the database insert and broadcasts the change via WebSockets. Active responder apps instantly refresh their AlertsScreen.
4. **Remote Alerting:** The application queries the profiles table for responder Push Tokens. A secure POST request is dispatched to the Expo Push API, which routes the alert through Google FCM (Android) or APNs (iOS) to wake up devices running in the background.

---

6. Installation & Deployment Guide

6.1 Prerequisites

- Node.js installed
- EAS CLI installed globally (npm install -g eas-cli)
- Expo Go application installed on mobile device (for UI testing)

6.2 Project Setup

Bash

```
# 1. Clone the repository
git clone <your-repo-link>
cd safetyapp

# 2. Install all dependencies using npm
npm install
```

6.3 Development Workflow (Live Reloading)

Due to Expo SDK 53 restrictions on native push notifications, a custom development client is required to test the full lifecycle.

Bash

```
# Build the development client (One Time)
npx eas build --profile development --platform android

# Start the local server
npm run start-clean
```

(Install the generated APK on your device. Saving files in VS Code will now live-reload the app).

6.4 Production Workflow (Defense Build)

To generate the final standalone APK for the thesis defense:

Bash

```
npx eas build --profile preview --platform android
```

---

7. Troubleshooting & Known Issues

7.1 "Network Request Failed" Error

- **Cause:** Supabase enforces an inactivity rule on free tiers, pausing the database, OR Android blocks raw URI uploads.
- **Fix 1:** Go to the Supabase dashboard (org: hotsos) and click "Resume Project".
- **Fix 2 (Images):** Ensure ProfileScreen.js is utilizing the Base64 array-buffer upload method rather than raw file:// URIs.

7.2 Component Import Errors

- **SafeAreaView:** Must be imported from react-native-safe-area-context, not the standard react-native library.

7.3 Infinite Loop / Maximum Update Depth

- **Cause:** Improper useEffect dependencies in HomeScreen.js.

- **Fix:** Ensure the dependency array includes [session?.user?.id] and implement a guard clause (if (expoPushToken) return;) to prevent continuous re-renders.

7.4 Database Reset Constraints

If foreign key constraints prevent user deletion during testing, execute the following SQL command in the Supabase editor:

SQL

```
TRUNCATE TABLE public.alerts CASCADE;
TRUNCATE TABLE public.profiles CASCADE;
DELETE FROM auth.users;
```